Scaling with R

Frédéric Logé
frederic.logemunerel@gmail.com

2025-10-17

ENSAE-ENSAI
Formation continue
(Cepe)

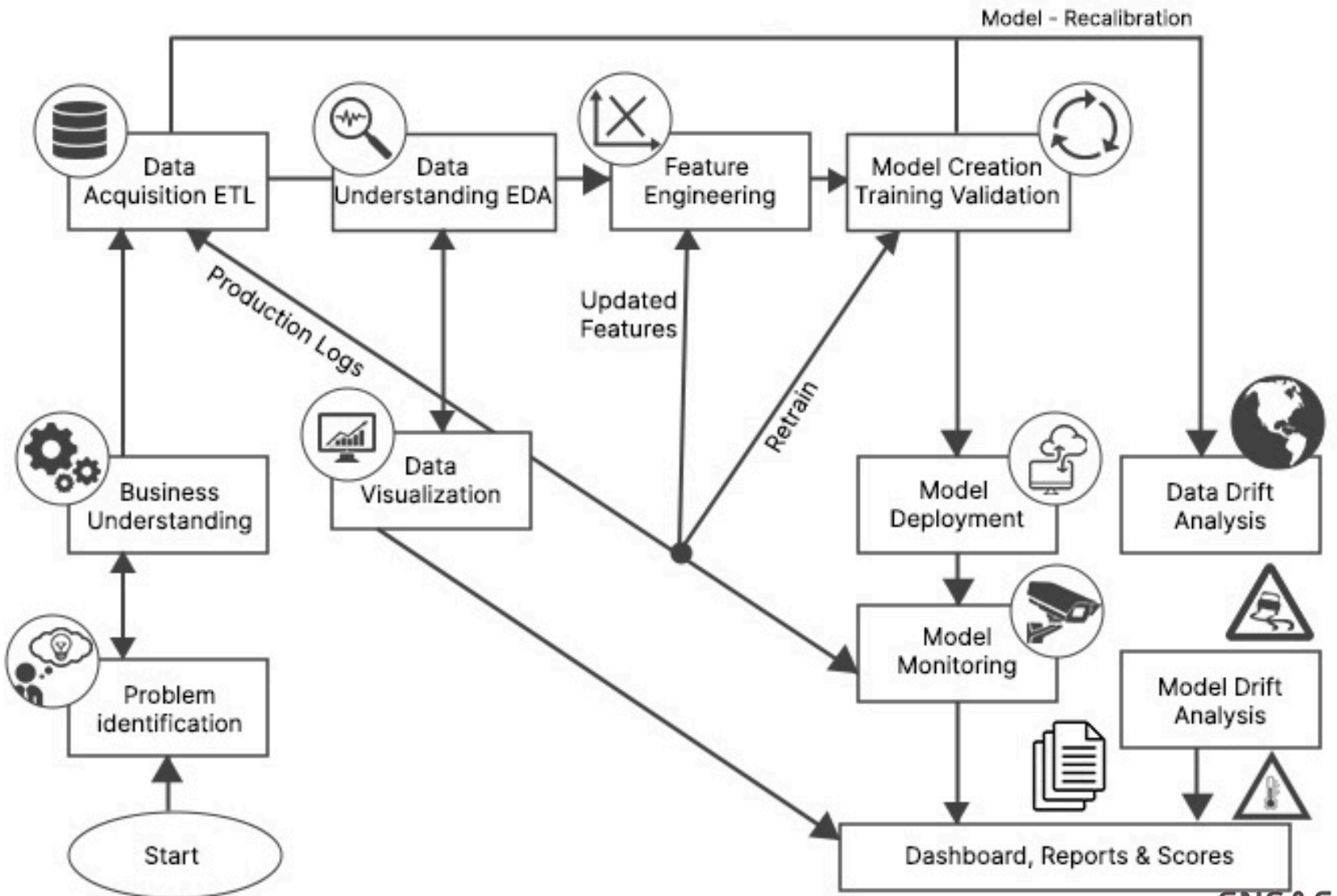
Table of contents

ENSAE-ENSAI
Formation continue
(Cepe)

Introduction

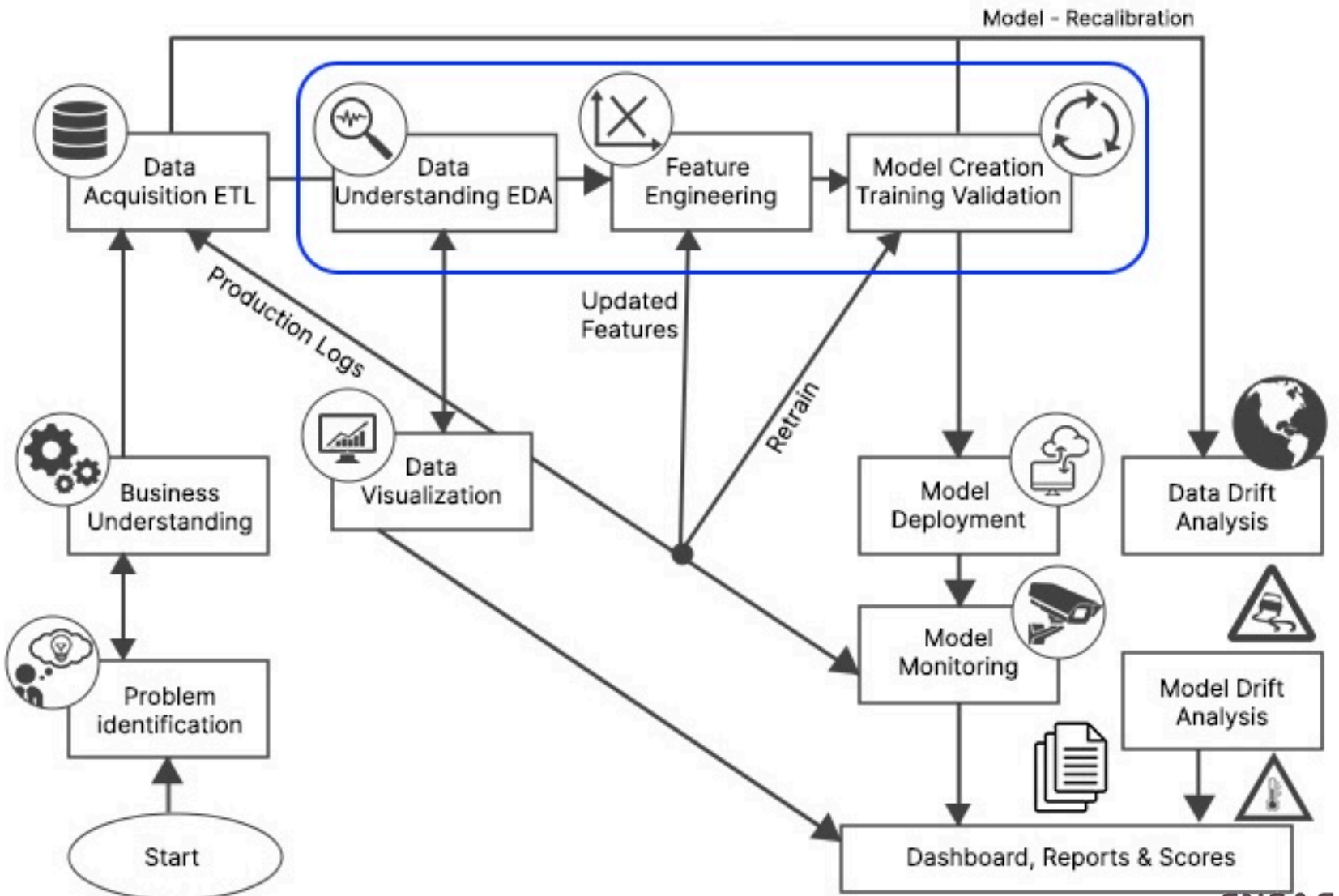
ENSAE-ENSAI
Formation continue
(Cepe)

Data Science Lifecycle



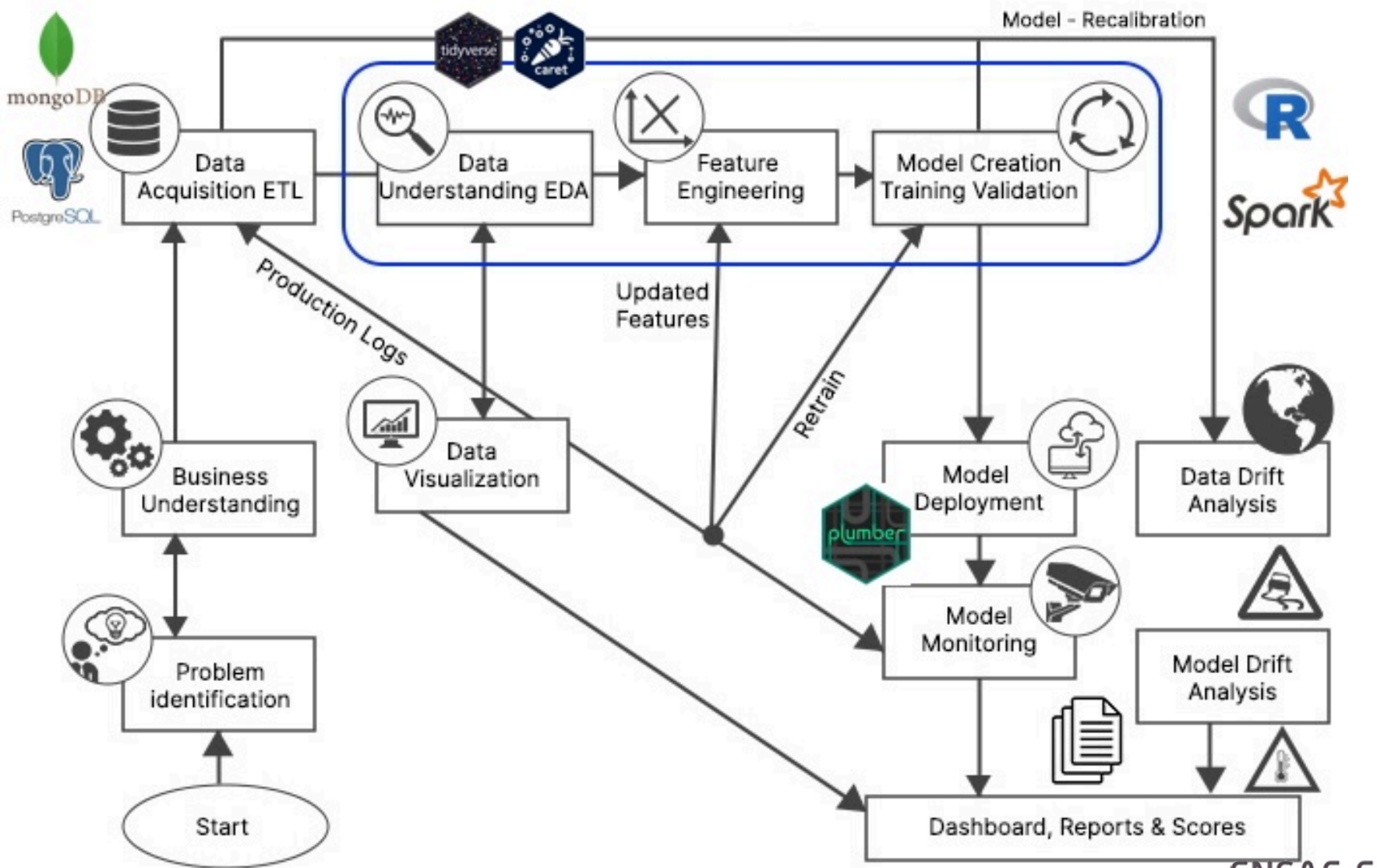
Data Science Life cycle, source

What are we going to work on today ?



Data Science Life cycle, source

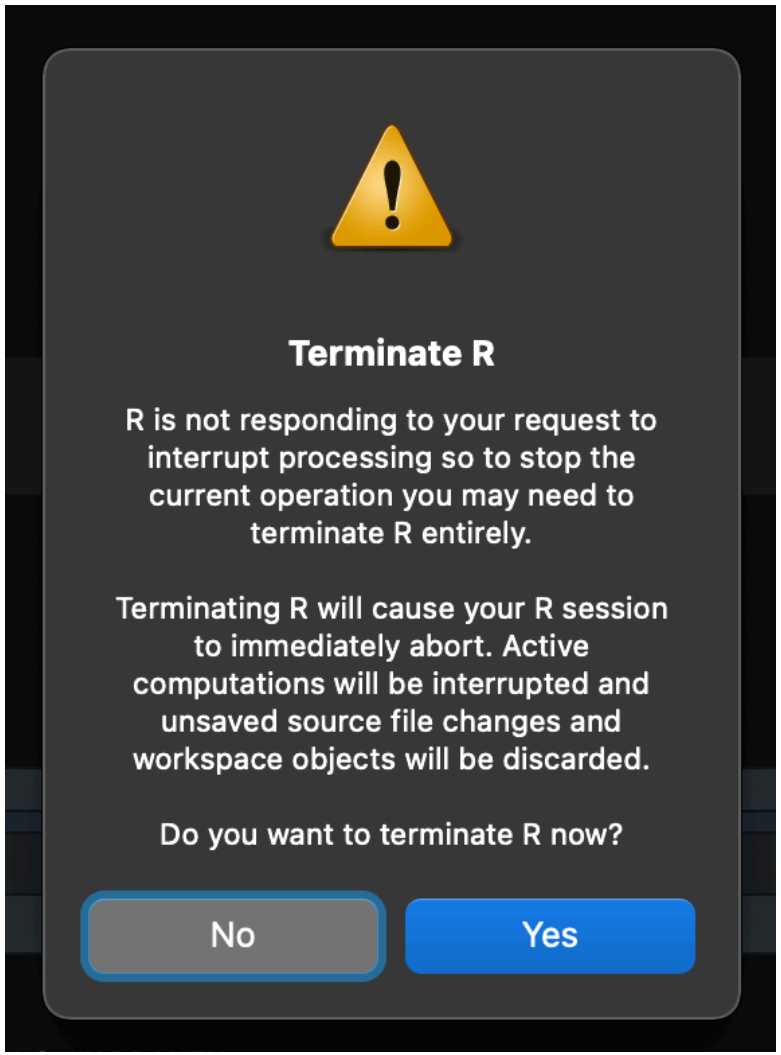
What are the tools involved ?



Data Science Life cycle, source

Common issues with large datasets

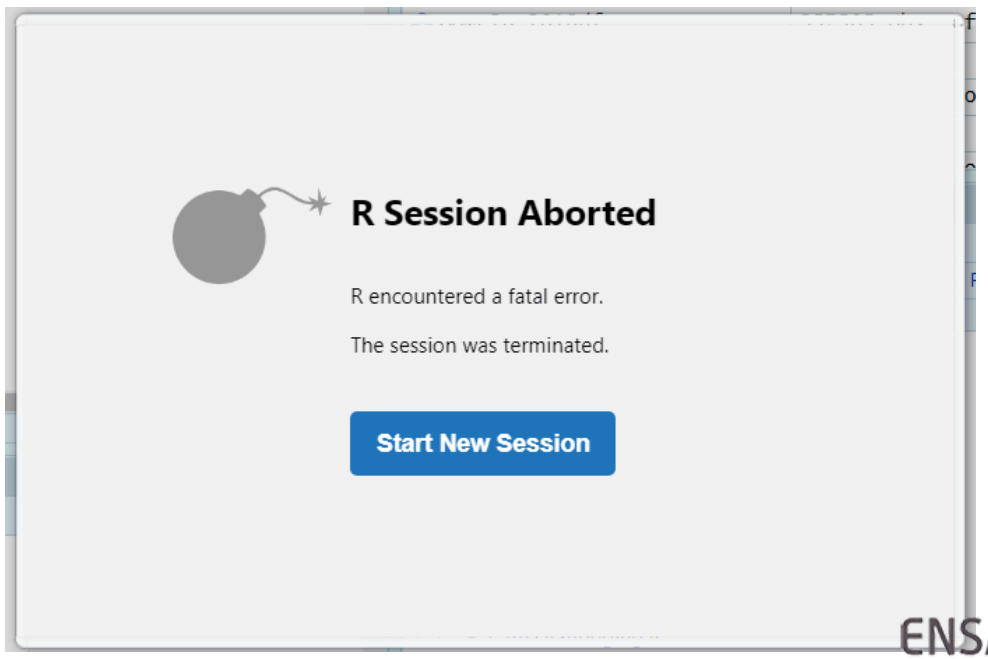
Time, Memory, Crash




```
* Found 1372 raster cells that were NA for one or more, but not all, predictor
* Removed 94 occurrence localities that shared the same grid cell.
* Removed 2 occurrence points with NA predictor variable values.
* Removed 1 background points with NA predictor variable values.
* Assigning variable bhutanlulc to categorical ...
* Clamping predictor variable rasters...
* Model evaluations with random 10-fold cross validation...

*** Running ENMeval v2.0.3 with maxnet from maxnet package v0.1.4 ***

  |====
r: cannot allocate vector of size 771.7 Mb
```



ENSAE-ENSAI
Formation continue
(Cepe)

But what can we do ?

A not exhaustive list of what we may do:

- **Optimize your R code** e.g. remove unnecessary loops, avoid data copy, loading unnecessary packages, etc, `Rcpp`
- **Rely on (most efficient) R packages** e.g. `dplyr`, `data.table`, `arrow`
- **Run code in parallel** e.g. `future` exploit hardware and distribute the work
- **Upgrade session's available memory** e.g. change RStudio config, get hardware update, setup virtual machine with higher memory config
- **Delegate treatment** e.g. `DBI`, `sparklyr`, `h2o` to perform operations with an engine more efficient than R
- **Work on data samples** e.g. active learning
- Breakdown tasks in your data science life cycle: **you cannot do everything in R**.

Writing efficient code

ENSAE-ENSAI
Formation continue
(Cepe)

Best R practice recommendations

- Cleanliness & tidyness: avoid data copy, avoid garbage names throughout code, treat your RAM with kindness
- Comments are mandatory, even if variable names are explicit
- Work under RProject
- If you wish to build something that should last (used by others, robust etc), developing as an R Package is mandatory. Not suitable when doing tutorials and trying a million different things though :/

Timing

When you are writing an R function, measuring it's execution time is good practice.

```r
# define a function
foo <- function(){ return(sum(1:1e6)) }

# measure execution of the function, once
# user: actual CPU time for the process
# system: any indirect operation due to the process: I/O of files, GC, memory allocation, ...
# elapsed: total elapsed time
system.time({ foo() })
```

```
   user  system elapsed 
      0       0       0 
```

```r
# repeat 50 times the function to get some statistics
microbenchmark::microbenchmark({ foo() }, times = 50)
```

```
Unit: nanoseconds
          expr min  lq     mean median  uq    max neval
 {     foo() } 164 164 15229.04    205 205 751530    50
```

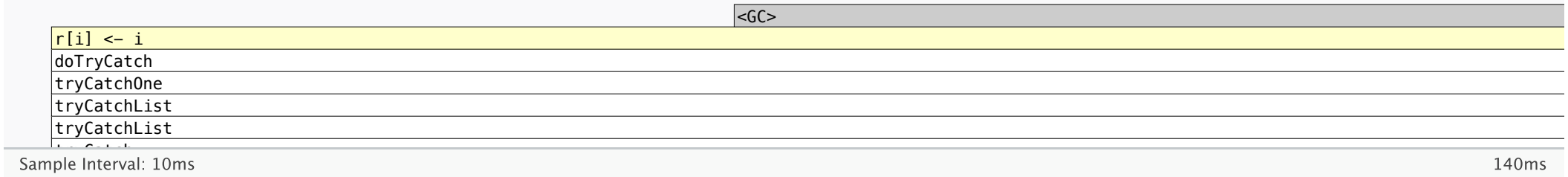
Profiling code

When your function (or general code) contains several instructions, profiling it allows you to see where exactly things go wrong. Here's a vanilla example:

```
# note that in RStudio you can use the "Profile" menu identically
profvis::profvis({
  r <- c()
  s <- 0
  for(i in 1:1e6){
    s <- s + i
    r[i] <- i
  }
})
```

Flame Graph | Data | Options

<expr>	Memory		Time	
1 `# note that in RStudio you can use the "Profile" menu identically`				
2 `profvis::profvis({`				
3 `  r <- c()`				
4 `  s <- 0`				
5 `  for(i in 1:1e6){`				
6 `    s <- s + i`				
7 `    r[i] <- i`	-29.4	41.0	140	
8 `  }`				
9 `})`				



Ex1: Moving Average

Moving Average

Input: x, numerical vector of length N

Output: y, numerical vector of same length, where $y_i := ((x_i - 1) + x_i + (x_i + 1))/3$ if $i > 1$ and $i < N$, otherwise y_i is NA.

Propose some implementations of this function and use the timing functions seen previously to time yourselves.

Solution in `exercise_1_moving_average.R`.

Some key learnings:

- Avoid dynamic memory allocation
- Avoid for loops

Why still create our own R functions ?

If it exists in R and ruins decently well, then why rebuild it ?

There are many cases were we might not find our pick:

- Data simulation purposes
- Statistical metrics
- Optimization functions

...

- Because we don’t know any other language 😭

Ex2: Dynamical System

Simulation

Consider the following dynamical system:

- x_0 is in $[0; 1]$
- λ is in $[0; 4]$

Then, for all $t \geq 1$:

$x_t = \lambda * x_{t-1} * (1 - x_{t-1})$.

Step 1. Code an R function which takes as input (x_0, λ, n) and returns output x_n.

Step 2. Run the function for a uniform grid of (x_0, λ), the size of your choosing.

Solution in `exercise_2_dynamical_system.R`

Key Learning: when you cannot remove the for loop, just code it in C++ 😉

See more info on this over here.

ENSAE-ENSAI
Formation continue
(Cepe)

Memory

Aside from time benchmarks, scanning our environment for large objects can always be useful.

- `ls()` provides you the list of elements in your env
- `object.size(<the element>)` gives you its size (also `lobstr::obj_size(<the element)`)
- `rm()` removes an object from the env
- `gc()` runs garbage collection (which runs periodically anyway so no need usually)

Why do we need garbage collection ?

- RHS data is created and bounded to one or more names
- When a modification on RHS is requested, a copy is made.
- When an element is removed from the environment, it removes the name and its bind to the value, but the value in memory is still taken, until the garbage collector does its job.

```r
a <- c(1, 2, 3)
b <- a # make copy

print(lobstr::obj_addr(a))
```

```
[1] "0x11ad70708"
```

```r
print(lobstr::obj_addr(b)) # wth ?? the same adress ?
```

```
[1] "0x11ad70708"
```

```r
b[1] <- 0

print(lobstr::obj_addr(a)) # value of a hasn't moved!
```

```
[1] "0x11ad70708"
```

```r
print(lobstr::obj_addr(b)) # but value of b has
```

```
[1] "0x11af66e98"
```

Ex3: Order of Magnitude

Order of Magnitude

Generate random datasets:

- vary the data types: numerical, boolean, categorical
- vary the number of observations and columns

For each dataset generated, compute the object size and make a nice visualization from it.

Solution in `exercise_3_memory_costs.R`

Key Learnings:

- Data storage of a variable in vectors is around 8MB/million
- Can drop if you rely on integer coding (integer, factor, boolean)
- R has specific internal mechanisms to handle efficiently memory when you copy variables

Ex4: Generate data under constraint

Generate IDs

Generate IDs with the following structure:

- an ID consists of 10 digits (0-9) chosen at random, a special character (@, é, è, ê) and 5 letters (lowercase or uppercase no matter)
- to be valid, the sum of the digits must be lower than 70 (rule 1)
- to be valid, you cannot have the letter e in the ID and a 0 or 9 in the digits (rule 2)

Your task is to find an efficient way to generate those IDs.

Solution can be found in `exercise_4_generate_IDs.R`.

Key takeaway:

- Vectorization is a great tool to write efficient code
- Even with individual constraints and unavoidable loops, you can significantly increase computation speed by designing how you handle your computations

Exercises to go further

- Write an efficient ifelse block statement taking as input a numerical vector, with the conditions of your choice
- Investigate memory examples at a lower scale
- Rcpp implementation of optimization function e.g. Decision Tree
- Implement exercise 4 using Rcpp

Parallel treatment

The `future` package

If you look for resources around parallel programming, you'll inevitably found the following names: `snow`, `parallel`, `foreach` and `future`. Those are not the only ones but clearly the most popular.

In all instances, the code looks somewhat (if not a little more complex) like the one we shall use from the `future` package:

```
1 library(future)
2 library(doFuture)
3
4 plan(multisession, workers=1) # <- only one CPU working
5 system.time({
6   x <- foreach(i = 1:4) %dofuture% {
7     Sys.sleep(2)
8   }
9 }) # ~8 seconds elapsed
```

```
   user  system elapsed
  0.065   0.014   8.095
```

```
1 plan(multisession, workers=4) # <- four CPU working!!
2 system.time({
3   x <- foreach(i = 1:4) %dofuture% {
4     Sys.sleep(2)
5   }
6 }) # ~2 seconds elapsed
```

```
   user  system elapsed
  0.179   0.009   2.108
```

Ex5: When should we use parallel ?

> **Parallel**
>
> To verify that you can indeed rely on parallel programming, let's do the following experiment: define a function that is quite slow to run and compare the runtime when using several CPUs vs only one. Do the same thing with a fast function. What can you conclude ?

Solution is in `exercise_5_parallel_experiments.R`.

Key takeaways:

- It is very easy (today) to require your code to be run asynchronously, which can speed up execution time tremendously (usually *4 but sometimes more depending on your hardware setup)
- Note that if your code runs very fast, then the runtime might not improve and even worsen, as now there is communication to the workers which did not occur before

Going further

In the futureverse, you also have `future.apply` which proposes a futurized version of base R apply and `furrr` a futurized version of purrr. `foreach` is already pretty great, it's more a matter of personal choice.

I strongly encourage you to go back to your personal/professional projects and see where you could setup such parallel treatments. Sometimes it's much harder to see where to apply this, than to actually apply it!

Data Wrangling

ENSAE-ENSAI
Formation continue
(Cepe)

Wrangling with `dplyr` and `tidyr`

Before diving into a large dataset, let's consider a mid-one and focus on dplyr verbs, which, **spoiler alert**, are used a lot.

`dplyr` and `tidyr` are two major packages of the `tidyverse` that allow for easy data manipulation and statistics computations based on tabular data. If you are not familiar with those packages, please check out the tutorial `tutorial_1_dplyr_tidyr_dtplyr.R` for an easy introduction to the main verbs.

Note: if you prefer data.table manipulations, which are more efficient on large data but syntatically maybe harder to apprehend, the dtplyr package handles most conversions from dplyr verbs to data.table verbs. At such, it is a great way to ease into such transitions.

How much data does one need ?

Before diving into treatment of very large datasets, I would like to clearly convey the following message: **it is quite likely that you only need a quite “small” sample of data to work, clearly not the full database**.

The rationale behind it is simply that data diversity is limited by the dimensions in which you work. So if you get more and more observations, at some point there is no more diversity in the additional data.

Don’t trust me ? Let’s put it to the test!

Ex6: How much data does one need ?

How much data does one need ?

Consider the `nafld1` dataset from `survival` package, for which we have several features available for individuals which have or not a specific disease. Your task is to build a prediction model of the disease and evaluate how the model performs depending on the training set size. Please ensure that the test set is large enough to have accurate performance metrics.

Solution is in `exercise_6_nafld1.R`.

In this particular case, about 1000 samples are sufficient to reach max performance. Note that, in practice, gradually increasing the sample size is good practice to avoid long runtimes.

Ex7: Reading large dataset

For the following, please go here and download the file locally.

Note: if this link is inactive some day, either pick any .csv file over 1Gb that you may have, or use the code in `generate_house_pricing_data.R`.

This dataset is 7.5Gb stored on disk. If you're a gambler, please try and read this in memory, see where that gets you 😈

> **Read a sample of the data**
>
> Let's try just importing a sample of the data, like 10000 rows. How would you go around doing it ?

Solution is in `exercise_7_reading_large_dataset.R`

Key takeaways:

- to import data in R: `data.table::fread` > `readr::read_csv` > `base::read.csv`, at least in big data setting
- you cannot read 7.5Gb of data in R's memory, so another way has to be taken!

Parquet, Apache Arrow and duckdb

First off, we'll convert the .csv to the parquet format, which stores information in binary in a column format. That way, if some columns are needed, only those are loaded. Second, with Apache Arrow and duckdb, the data will not be loaded in memory, operations will be executed directly on the disk data.

Aside from being able to read the data, this allows us to apply either `dplyr` verbs (using `arrow`) or SQL (using `duckdb`) for common data analytics.

See `tutorial_2_parquet_arrow_duckdb.R` around those topics, or the next slides. A comprehensive benchmark of the solutions is available here.

ENSAE-ENSAI
Formation continue
(Cepe)

Apache Arrow

damn, can this get any easier ?

```r
# read parquet
# <!> as_data_frame=FALSE ensures you don't read it in memory
house_parquet <- read_parquet(file="data/local_parquet.parquet", as_data_frame = FALSE)

# check out content
house_parquet

# build a query using common base/dplyr verbs
req <- house_parquet %>%
  select(price, lat, long) %>%
  filter(lat > 0.5) %>%
  summarise(mean(price))

# check the request content - nothing is run yet
print(req)

# actually run the query **on the whole database** and locally collect the result
req |> collect()

# note: if you need to just calculate a temporary result, and not collect it, use the `compute` meth
```

ENSAE-ENSAI
Formation continue
(Cepe)

duckdb

still using `dplyr` verbs !!

```r
# prepare DB connection
con <- dbConnect(duckdb())

# setup duck db connection
# you can also read directly the parquet without relying on the previous read_parquet
house_parquet_tbl <- to_duckdb(house_parquet, con, "house_parquet_con")

# same code as before
# -> this runs the query BUT the result is not stored
req <- house_parquet_tbl %>%
  select(price, lat, long) %>%
  filter(lat > 0.5) %>%
  summarise(mean(price))

# run and export the value
res <- req |> collect()
```

Going further

`duckdb` and `arrow` are popular solutions, but not the only ones. I encourage you to search for the best option in your respective working environments.

Note that the data format is also of importance: we relied on parquet data since we had tabular data with a lot of records and a few columns. Another storing format is `avro`, where record a stored in binary format, per row, to facilitate this querying.

If you want to look into working with databases in R (quite similar topics to what has been seen already), you have `tutorial_3_db.R`.

Statistical Analysis

In this section, we will dive into two R packages allowing us to leverage high-end distributed frameworks for statistical analysis: `h2o` and `sparklyr`.

ENSAE-ENSAI
Formation continue
(Cepe)

Platform h2o

`h2o` is a distributed platform for ML. You can download and run it on your local machine, but you could also deploy it on a server. The backend code is in Java and therefore runs super fast, it is particularly interesting at scale. See the tutorial `tutorial_4_h2o.R` and check out how it works (broadly) over here:

```
train_h2o_data <- as.h2o(...) # <- give some data

h2o.init() # start h2o local JAVA server

auto_ml <- h2o.automl(y=..., training_frame=train_h2o_data) # run auto ML

best_model <- h2o.get_best_model(auto_ml) # extract best model

prediction <- h2o.predict(best_model, test_h2o_data) # check out predictions
```

Apache Spark

Apache Spark allows for distributed ML and analytics on data. You can work on Onyxia or in a local spark server to run the code in `tutorial_5_sparklyr.R`.

The process looks something along the following lines:

```r
library(sparklyr) # load lib

sc <- spark_connect(master = "local") # connect to the server, local or not

flights_tbl <- copy_to(sc, nycflights13::flights, "flights") # copy data

model_delay <- flights_tbl %>%
  na.omit(arr_delay, distance) %>%
  ml_linear_regression(x=., formula=arr_delay~distance) # perform ML on the cluster
```

Going further

You are very much encouraged to apply those methods to your own projects and see what works and what doesn't. As we've seen in the past chapters, there is no one solution and no solution fits all.

Something quite interesting would be to compare runtime when running usual R code versus those solutions. Happy exploration!

Conclusion

Elements of conclusion

Here's some key learnings:

- Write efficient code: measure time and memory, rely on vectorization and base functions, code in C++
- Use appropriate packages depending on the task at hand e.g. `arrow`
- Remember: you don't need all the data for what you wish to accomplish. Don't be greedy for nothing!

Resources

Textbooks

Advanced R, 2nd Edition, utilitR book, Advanced R training

Benchmarks

Data Wrangling Benchmark 1, Data Wrangling Benchmark 2

Cheatsheets & documentation links

data.table, dtplyr, sparklyr

Miscellaneous

SAS to R

ENSAE-ENSAI
Formation continue
(Cepe)